Scale Reader — Hardware Integration Guide

Overview

Scale Reader connects to a weight indicator (or microcontroller acting as one) via a USB/serial connection. The PC reads weight data from the device and optionally sends a relay control signal back when a load is captured.

Serial Port Settings

Parameter	Value
Baud rate	115200
Data bits	8
Stop bits	1
Parity	None
Line ending	LF (`\n`) — CR+LF will not work

Select the COM port in the app's **Settings** tab and click **Connect**.

Data the App Expects to Receive

Each line must contain exactly **2 comma-separated fields** followed by a newline (`\n`). Lines with fewer than 2 fields are silently ignored.

```
<weight>,<units>\n
```

Fields

#	Name	Type	Expected values	Notes
0	Weight	float	e.g. `1234.5`	Decimal separator must be a period (`.`). No thousands separator.
1	Units	string	`lb` or `kg`	Tells the app how to interpret the weight value. The display unit can be overridden in the Settings tab independently.

> **Stability is handled internally.** The app compares consecutive weight readings against a configurable tolerance window. There is no need to send a stability flag from the microcontroller.

Example Lines

```
1250.0,lb
34725.0,kg
```

Auto-Weigh Behaviour

When **Auto Weigh** is enabled the app runs a state machine on every received line:

```
[Empty] --(weight >= min AND interval elapsed)--> [Settling]
   ^                                                  |
   | weight drops below min                           | weight drops below min
   |                                                  v
   +------------------------------------------------ [Captured]
                                                      |
                                status == "Stable" for stability seconds
                                                      |
                                                      v
                                              Record load & send RELAY:1
```

Setting	Description
Min Weight	Weight threshold (in the chosen display unit) before a truck is watched
Stability (s)	Seconds the indicator must continuously report `Stable` before a capture is made
Signal (s)	How long `RELAY:1` stays active after a capture
Min Interval (s)	Lockout period after a capture before the next truck can be detected. Set to 0 to disable.

Data the App Sends Back

After a load is captured the app writes one of two ASCII commands to the serial port:

Command	When sent
`RELAY:1\n`	Immediately after a successful capture
`RELAY:0\n`	After the Signal timer expires

The microcontroller can use these to drive a relay, buzzer, indicator light, or any other output that confirms a weighing has been recorded.

Microcontroller Example — Arduino / Nano / Uno

```
#define RELAY_PIN 7

void setup() {
    Serial.begin(115200);
    pinMode(RELAY_PIN, OUTPUT);
}

void loop() {
    // --- Send weight data ---
    float weight = readScaleKg();    // replace with your ADC / HX711 read

    Serial.print(weight, 1);         // field 0 - weight
    Serial.print(",kg");             // field 1 - units
    Serial.print("\n");              // LF only - do NOT use Serial.println()

    // --- Handle relay commands from PC ---
    if (Serial.available()) {
        String cmd = Serial.readStringUntil('\n');
        cmd.trim();
        if      (cmd == "RELAY:1") digitalWrite(RELAY_PIN, HIGH);
        else if (cmd == "RELAY:0") digitalWrite(RELAY_PIN, LOW);
    }

    delay(200);   // ~5 readings per second is sufficient
}
```

Microcontroller Example — ESP32

```
#define RELAY_PIN 23

void setup() {
    Serial.begin(115200);
    pinMode(RELAY_PIN, OUTPUT);
}

void loop() {
    float weight = readScaleKg();

    Serial.printf("%.1f,kg\n", weight);

    if (Serial.available()) {
        String cmd = Serial.readStringUntil('\n');
        cmd.trim();
        if      (cmd == "RELAY:1") digitalWrite(RELAY_PIN, HIGH);
        else if (cmd == "RELAY:0") digitalWrite(RELAY_PIN, LOW);
    }

    delay(200);
}
```

Tips

- **Send rate** — 5–10 lines per second is plenty. The stability timer is measured in whole seconds so faster rates do not improve accuracy.
- **Units field** — The app stores all weights internally in kg regardless of the units field. Send whatever unit your sensor produces and set the matching field. The display unit in the app is configured separately in Settings and does not affect what you send.
- **Line ending** — Use `\n` (LF only). Arduino's `Serial.println()` sends `\r\n` (CR+LF) which will break parsing. Use `Serial.print("\n")` instead.
- **No relay needed** — If you do not need the relay output, simply ignore anything received on the serial port. The app will not error if `RELAY` commands go unhandled.